

CSE574: Cloud Computing

Indian Institute of Technology (BHU) Varanasi

Assignment 2: Creating an Ubuntu Virtual Machine

Instructor: Dr. Prasenjit Chanak

Name: Jayesh Krishan Puri
Branch: Mining Engineering, 3rd Year B.Tech
Roll No: 24155058
Date: 18 August 2026
GitHub <https://github.com/Jayesh-Dev21>

1. Host Machine Specifications

The assignment was completed on a personal laptop running **CachyOS** (an Arch Linux-based rolling-release distribution) with **KDE Plasma** as the primary desktop. A secondary bare-metal Arch Linux installation with the **i3 tiling window manager** is also maintained for testing new kernel releases and patches.

OS	CachyOS (Arch Linux-based, rolling release)
CPU	x86_64, hardware virtualisation enabled
RAM	16 GB
Storage	500 GB NVMe SSD
Hypervisor	QEMU/KVM + libvirt (virt-manager)

Since QEMU/KVM is the native Linux hypervisor and does not require out-of-tree kernel modules (unlike VirtualBox on a rapidly-updating Arch system), it was chosen over Oracle VirtualBox. The core objective of the assignment – creating a VM with specified resources and installing Ubuntu – is fully met.

2. VM Configuration

The virtual machine was created using **virt-manager** with the following configuration, satisfying the assignment requirements:

VM Name	ubuntu Server
RAM	2 GB (2048 MiB)
vCPUs	2 (host-passthrough mode)
Disk	20 GB, VirtIO controller
Network	VirtIO NIC, libvirt NAT (default network)
Display	SPICE + VirtIO GPU
Chipset	Q35, QEMU 10.2
Hypervisor	QEMU/KVM (hardware-assisted)
OS (Guest)	Ubuntu Server 26.04
Login User	notadmin / 123456

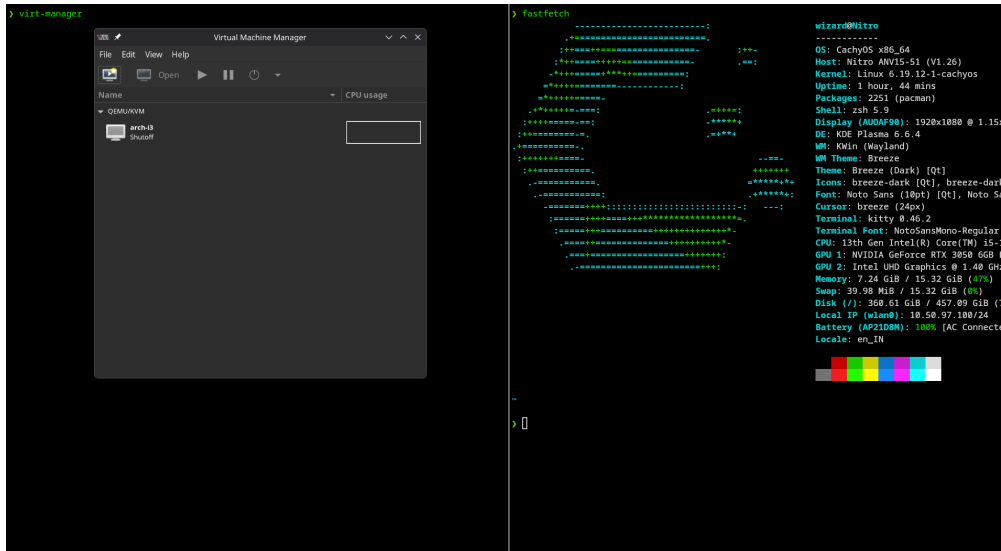


Figure 1: virt-manager hardware summary – 2 GB RAM, 20 GB disk, VirtIO devices

3. Installation Steps

3.1.1. Step 1: New VM and ISO Selection

In virt-manager, “New Virtual Machine” was clicked and “Local install media” selected. The Ubuntu Server 26.04 ISO was loaded from the local filesystem.

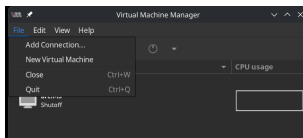


Figure 2: New VM wizard

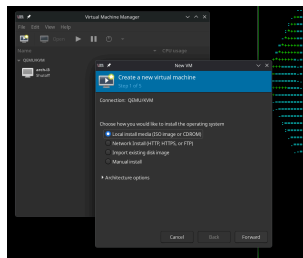


Figure 3: Local ISO selection

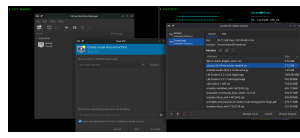


Figure 4: OS type selection

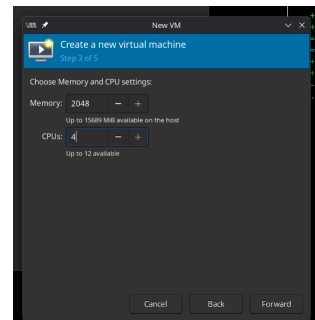


Figure 5: 2 GB RAM, 2 vCPUs

3.1.2. Step 2: Disk and VM Name

A 20 GB virtual disk was created. The VM was named “ubuntu Server”.

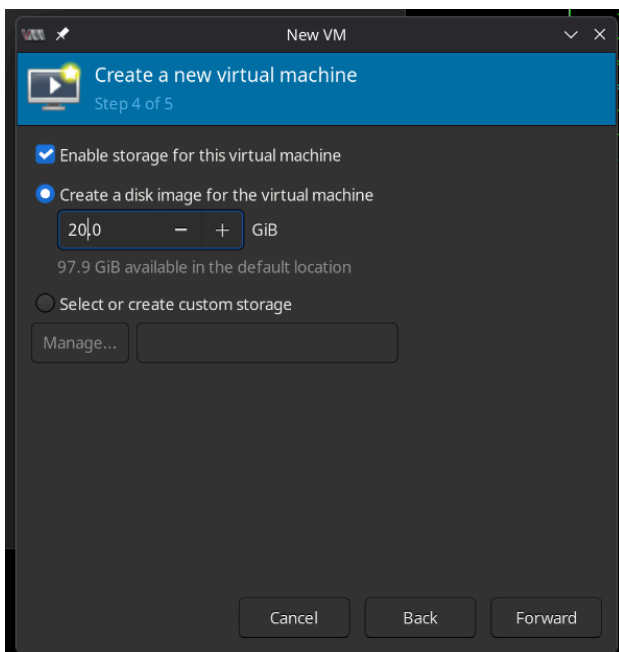


Figure 6: 20 GB virtual disk

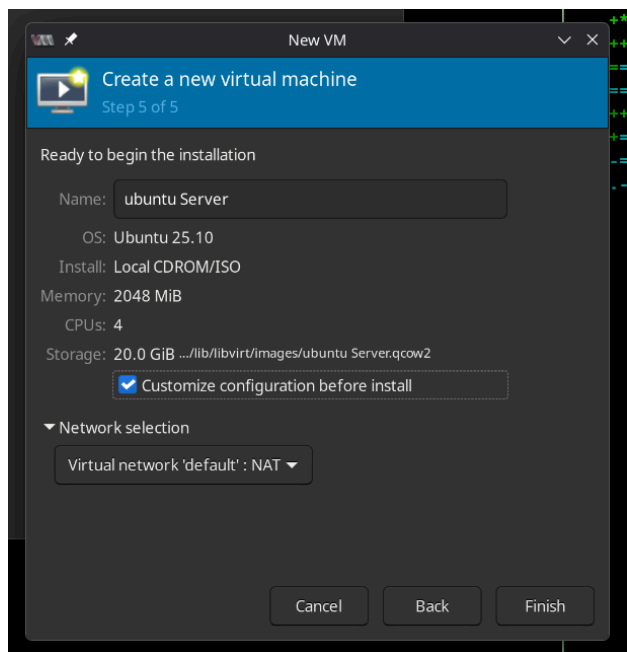


Figure 7: VM name and final summary

3.1.3. Step 3: Boot and Installer Launch

The VM booted from the ISO. GRUB appeared, then the Ubuntu Subiquity installer loaded.

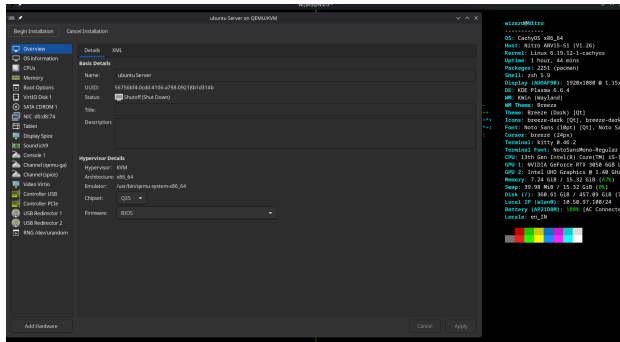


Figure 8: GRUB boot menu

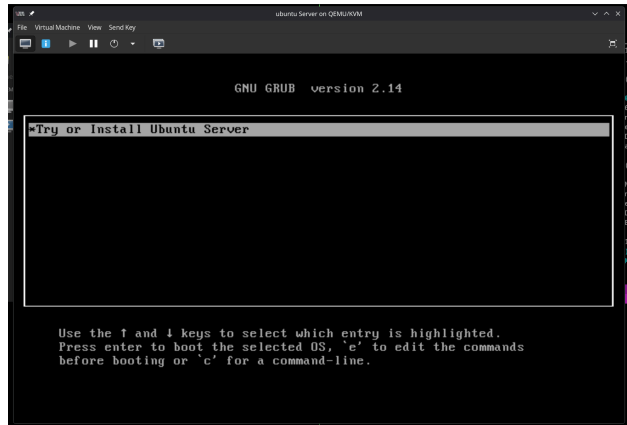


Figure 9: VM startup from ISO

3.1.4. Step 4: Boot Log and Installer Welcome

The kernel enumerated virtual hardware (VirtIO disk, NIC, USB). The Subiquity installer loaded and showed the welcome/language screen.

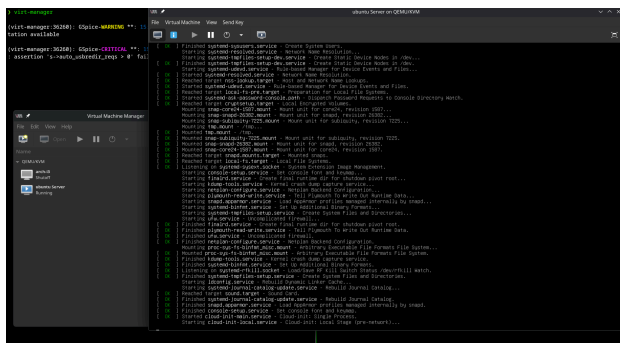


Figure 10: Initial boot/hardware log

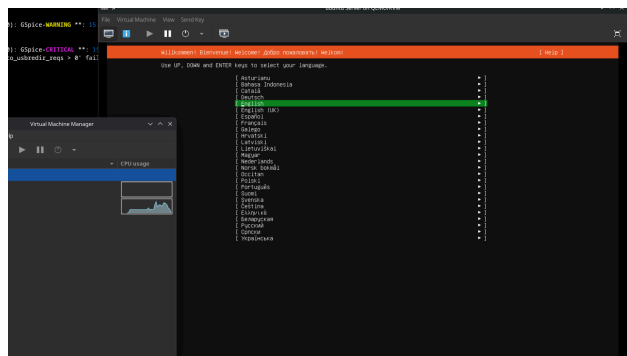


Figure 11: Installer welcome screen

3.1.5. Step 5: Storage and Profile Configuration

The 20 GB VirtIO disk was detected. Default guided LVM partitioning was selected. A user profile was created: username **notadmin**, password **123456**, hostname configured.

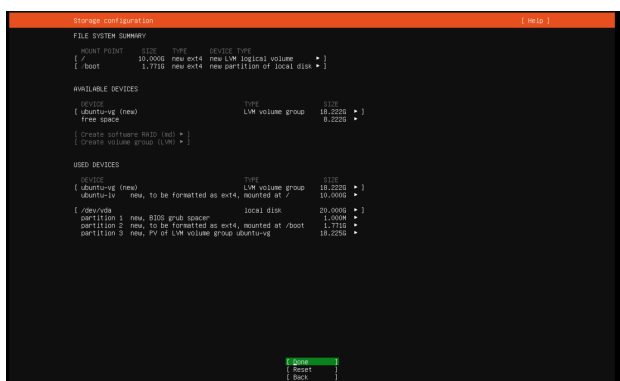


Figure 12: Storage layout – 20 GB LVM

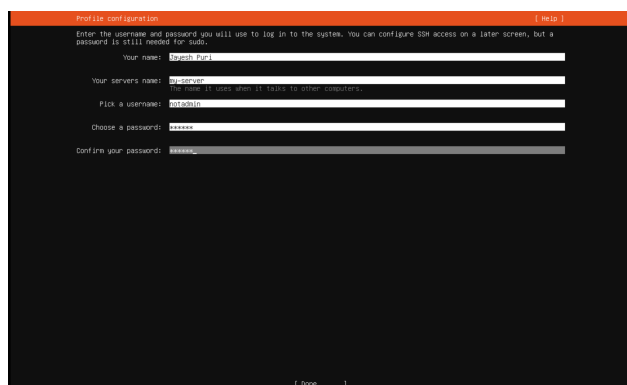
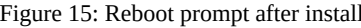


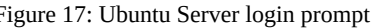
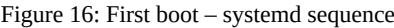
Figure 13: User profile setup

3.1.6. Step 6: Installation and Reboot

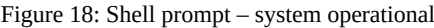
Packages were installed. On completion, the installer prompted to reboot. The virtual CDROM was ejected and the VM rebooted.



Ubuntu Server booted from the installed disk, ran systemd initialisation, and presented the login prompt. Login was successful with the configured credentials.



Post-login, the shell was accessible confirming a working install. The VM was then cleanly shut down using `sudo shutdown -h now`.



No significant challenges were encountered during the installation. Since the virtualization stack was already well-configured with SPICE for display, VirtIO paravirtualised drivers for disk and network, and libvirt managing the KVM backend, the entire setup from VM creation to a working Ubuntu Server login went smoothly without any issues.

QEMU/KVM uses hardware virtualisation extensions so most guest code runs directly on the CPU with minimal overhead.

Boot time was roughly 12-15 seconds, close to a bare-metal Ubuntu Server install.

Memory: The 2 GB fixed allocation was sufficient for Ubuntu Server. The virtio-balloon device allows the hypervisor to reclaim unused guest memory when the VM is idle, which is more efficient than static allocation in VirtualBox.

Disk: The guest and host communicate via shared ring buffers, bypassing the overhead of emulating a physical SATA/IDE controller. With the VirtIO driver, disk performance inside the VM approached native NVMe speeds.